Санкт-Петербургский государственный университет
аэрокосмического приборостроения

КАФЕДРА № 41

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

Старший преподаватель		**М В Загураева**
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ

Лабораторная работа № 4

по курсу: Информатика

РАБОТУ ВЫПОЛНИЛ

Студент гр. № 8626в		**К К Журавлев**
	20.10.2026	
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

Лабораторная работа № 4

1 Цель работы

Получение практических навыков применения современных технологий искусственного интеллекта (больших языковых моделей и ИИ-агентов) для разработки, проверки и доработки web-сайтов.

2 Постановка задачи

Для выполнения индивидуального задания в соответствии с методическими указаниями я выбрал практическую производственную тему — разработку внутреннего веб-сервиса интерактивной диагностики и ремонта термопластавтоматов (ТПА) для предприятия ООО «Дельфин Групп» в Санкт-Петербурге.

Основные параметры проекта:

– *Название проекта:* веб-сервис диагностики и ремонта термопластавтоматов «ТПА-Мастер: Дельфин Групп».

– *Предметная область:* промышленное производство пластиковой тары и упаковки методом скоростного литья под давлением, техническое обслуживание и ремонт термопластавтоматов (ТПА) и пресс-форм в ООО «Дельфин Групп» (Санкт-Петербург, п. Металлострой, дорога на Металлострой, д. 5, лит. П).

– *Целевая аудитория:* сменные мастера, наладчики ТПА, инженеры КИПиА и слесари-ремонтники литьевого цеха, которым нужен чёткий и быстрый инструмент без лишних элементов, чтобы прямо у станка с планшета, смартфона или цехового ПК отметить возникшие сбои и получить готовую инструкцию по ремонту.

– *Основная цель сайта:* предоставить мастеру единую интерактивную базу классических и нетипичных неисправностей ТПА, где при выборе всех текущих проблем станка автоматически формируется полное пошаговое решение по устранению сбоя, перечень нужного инструмента и рабочие допуски параметров литья.

– *Основные действия пользователя:* выбор серии ТПА или узла станка, фильтрация ошибок («Классические ошибки литья и ТПА» / «Аппаратные и нетипичные сбои»), отметка чекбоксами всех наблюдаемых симптомов, получение сводной пошаговой инструкции по ремонту, отметка выполненных шагов в сменном журнале и вызов ремонтной бригады.

– *Используемые технологии:* HTML5, CSS3 (с подключением фреймворка Tailwind CSS через CDN для строгой индустриальной сетки), шрифты Google Fonts (Inter для текста

и JetBrains Mono для кодов ошибок и параметров давления/температуры), чистый JavaScript (ES6) и localStorage для сохранения выбранных ошибок и журнала смены в браузере.

3 Исходный промпт для генерации сайта и его оценка

3.1 Оценка первоначального базового промпта по схеме R-T-F-C

На первом этапе я сформулировал простой черновой запрос для создания сайта диагностики термопластавтоматов и проверил его по структуре R-T-F-C (Role, Context, Task, Format, Constraints):

– *Role (Роль):* отсутствует. В первоначальном запросе не было указано, что модель должна выступать в роли опытного frontend-разработчика промышленных интерфейсов (HMI/Service UI) со знанием устройства ТПА.

– *Context (Контекст):* прописан слишком кратко. Было указано только «сайт по ошибкам ТПА», но не уточнялась специфика предприятия ООО «Дельфин Групп» (скоростное литьё тонкостенной полипропиленовой тары, ремонт пресс-форм и ТПА в Металлострое) и требования к лаконичному рабочему интерфейсу для сменного мастера.

– *Task (Задача):* описана только общая идея (выбор ошибок и вывод инструкции), без разделения на классические дефекты литья (недолив, облой, утяжины, пригар) и аппаратные поломки узлов (смыкание, впрыск, гидравлика, нагрев, пресс-форма), а также без точного списка секций страницы.

– *Format (Формат):* указан чётко — выдать полный рабочий код (HTML, CSS и JavaScript) в одном файле index.html.

– *Constraints (Ограничения):* указаны базовые требования: строгая контрастная палитра без отвлекающего декора, удобство нажатия на чекбоксы с мобильного устройства или планшета в цеху и плавная навигация.

3.2 Доработанный промпт и выбранная модель

Для генерации сайта я использовал большую языковую модель Google Gemini. Сначала я попросил модель помочь составить структурированный шаблон промпта по схеме R-T-F-C, после чего дополнил его реальными данными компании ООО «Дельфин Групп» (г. Санкт-Петербург, п. Металлострой, дорога на Металлострой, д. 5, лит. П), перечнем классических и нетипичных неисправностей ТПА и логикой интерактивного конструктора решений по выбранным чекбоксам. Сторонний скриншот-референс я не прикладывал, так как детально задал структуру индустриального интерфейса в тексте запроса.

Итоговый текст промпта, который я отправил в новом диалоге:

Сначала я обратился к большой языковой модели с запросом сгенерировать для меня шаблон промпта: «Мне нужно создать одностраничный сайт-сервис для мастера цеха по диагностике и ремонту термопластавтоматов (ТПА). Создай промпт для LLM по схеме R-T-F-C, чтобы при выборе текущих ошибок ТПА мастер сразу получал полную пошаговую инструкцию по починке. Мне нужен один HTML-файл с CSS и JS». Полученный шаблон я проверил и дополнил конкретными узлами ТПА, классическими и сложными ошибками литья и реквизитами ООО «Дельфин Групп».

Role: Ты опытный Senior Frontend-разработчик и UX/UI-проектировщик промышленных инженерных систем (Industrial HMI & Maintenance Web Apps), отлично разбирающийся в устройстве, наладке и ремонте термопластавтоматов (ТПА) и пресс-форм.

Context: Нужно разработать внутренний одностраничный веб-сервис «ТПА-Мастер: Дельфин Групп» для производственной и сервисной службы компании ООО «Дельфин Групп» (г. Санкт-Петербург, п. Металлострой, дорога на Металлострой, д. 5, лит. П; тел. +7 (812) 464-87-12, info@delgroup.su). Предприятие более 20 лет занимается скоростным литьём тонкостенной полипропиленовой тары (банки, пресервные шайбы, контейнеры с IML-этикеткой), а также ремонтом пресс-форм и термопластавтоматов. Целевая аудитория сайта — сменные мастера, наладчики ТПА и инженеры по ремонту. Главная задача интерфейса: максимальная чёткость, наглядность и отсутствие «воды», чтобы мастер у станка мог отметить галочками (чекбоксами) все проблемы, которые сейчас случились на ТПА (как классические ошибки литья, так и аппаратные/нетипичные сбои), и мгновенно получить сводное пошаговое руководство по устранению неисправностей.

Task: Создай современный, строгий и полностью адаптивный одностраничный веб-сайт со следующей структурой и логикой работы:

1. Фиксированная шапка (Header): логотип и название «ТПА-Мастер | ООО Дельфин Групп», быстрые ссылки на разделы (Диагностика ТПА, Решение и инструкция, База узлов и допусков, Журнал смены, Сервис и контакты), индикатор статуса цеха и кнопка «Вызвать дежурного наладчика».

2. Главный рабочий экран (Hero / Панель быстрого старта): чёткий заголовок «Интерактивная диагностика и регламент ремонта ТПА», краткая сводка по парку оборудования цеха (скоростные ТПА для тонкостенной тары, стандартные гидравлические ТПА 120–500 т, участок ремонта пресс-форм), кнопки быстрого перехода к выбору ошибок и выбору типа ТПА.

3. Раздел «Интерактивный выбор проблем ТПА» (Ключевой функциональный блок): панель выбора модели/узла ТПА, строка быстрого поиска по симптому или коду ошибки и переключение вкладок категорий («Все проблемы», «Классические ошибки литья и ТПА», «Нетипичные и аппаратные сбои», «Пресс-формы и охлаждение»). Реализуй минимум 8 детальных карточек проблем с чекбоксами (можно выбрать сразу несколько проблем одновременно): классические ошибки — (1) Недолив тонкостенного изделия (падение давления/скорости впрыска, засор сопла, малая доза или низкая температура расплава ПП); (2) Облой по линии разъёма формы (падение усилия смыкания, превышение давления подпитки, износ запирающих кромок пресс-формы); (3) Утяжины и коробление геометрии банки/крышки (неравномерное охлаждение, засор каналов чиллера, малое время выдержки под давлением); (4) Пригар («эффект дизеля»), чёрные точки и серебристые полосы (защемление воздуха в выпарах формы, деструкция полимера, влажность сырья); аппаратные и нетипичные сбои узлов ТПА — (5) Не развивается усилие смыкания / перекос плит (срабатывание защиты формы низкого давления, износ пальцев КРМ, сбой линейки положения); (6) Прокручивание шнека без набора дозы / нестабильная подушка расплава (износ запорного кольца — торпеды, обрыв термопары ТХК или пробой кольцевого ТЭНа); (7) Падение давления в гидросистеме и перегрев масла выше 55 °C (залипание пропорционального клапана давления/потока, засор маслоохладителя, кавитация насоса); (8) Заклинивание толкателей или повреждение формообразующих пресс-формы (попадание инородного тела в расплав, износ направляющих колонок и втулок, сбой пневмосдува).

4. Раздел «Полное решение и пошаговая инструкция по ремонту» (Динамический блок): как только мастер отметил одну или несколько ошибок чекбоксами, этот блок автоматически перестраивается и выводит сводный регламент ремонта для всех выбранных проблем: уровень критичности, обязательные меры техники безопасности (блокировка LOTO, сброс давления, отключение нагрева), список необходимого инструмента, пошаговый алгоритм действий (Шаг 1, Шаг 2, Шаг 3... с интерактивными галочками выполнения каждого шага) и таблицу нормативных технологических параметров. Добавь кнопки «Сбросить выбор», «Сохранить ремонт в журнал смены» и «Скопировать инструкцию».

5. Раздел «Справочник узлов ТПА и регламент ТО ООО Дельфин Групп»: наглядные карточки 4 ключевых узлов (Узел впрыска и пластикации, Узел смыкания и КРМ, Гидростанция и охлаждение, Пресс-форма и горячий канал) с нормативами зазоров, температур расплава полипропилена (210–250 °C), давления и графиком ежесменного осмотра.

6. Раздел «Сменный журнал мастера»: таблица-лог последних диагностированных проблем на станках цеха (номер ТПА, выбранные ошибки, статус устранения, время) с сохранением в localStorage браузера.

7. Раздел «Сервисная служба и контакты» + Footer: реквизиты производственной площадки ООО «Дельфин Групп» (196641, Санкт-Петербург, п. Металлострой, дорога на Металлострой, д. 5, лит. П, тел. +7 (812) 464-87-12, +7 (812) 464-74-68, info@delgroup.su), график работы ремонтного участка и модальное окно быстрой заявки на ремонт ТПА или реставрацию пресс-формы.

Format: Предоставь весь код (HTML5, Tailwind CSS через CDN и чистый JavaScript с полной базой ошибок и инструкций) строго в одном файле index.html без сокращений и комментариев вида «// остальной код».

Constraints: Используй строгую индустриальную цветовую схему с высокой контрастностью для цехового освещения (тёмно-синий/графитовый #0F172A, стальной серый #1E293B, светлый фон рабочих панелей #F8FAFC, сигнальный оранжевый #EA580C и синий #2563EB для акцентов, красный #DC2626 для аварийных предупреждений). Без лишних декоративных картинок — используй чёткие SVG-иконки узлов, крупные кликабельные чекбоксы, читаемую типографику (Inter + JetBrains Mono), плавную прокрутку и полную адаптивность для планшета, смартфона и монитора ПК.

4 Проверка работы сайта и сравнение двух моделей

Полученный от модели код я скопировал в файл index.html в редакторе Visual Studio Code и запустил локальный сервер через расширение Live Server. Внешний вид главной страницы сервиса после запуска представлен на рисунке 1.

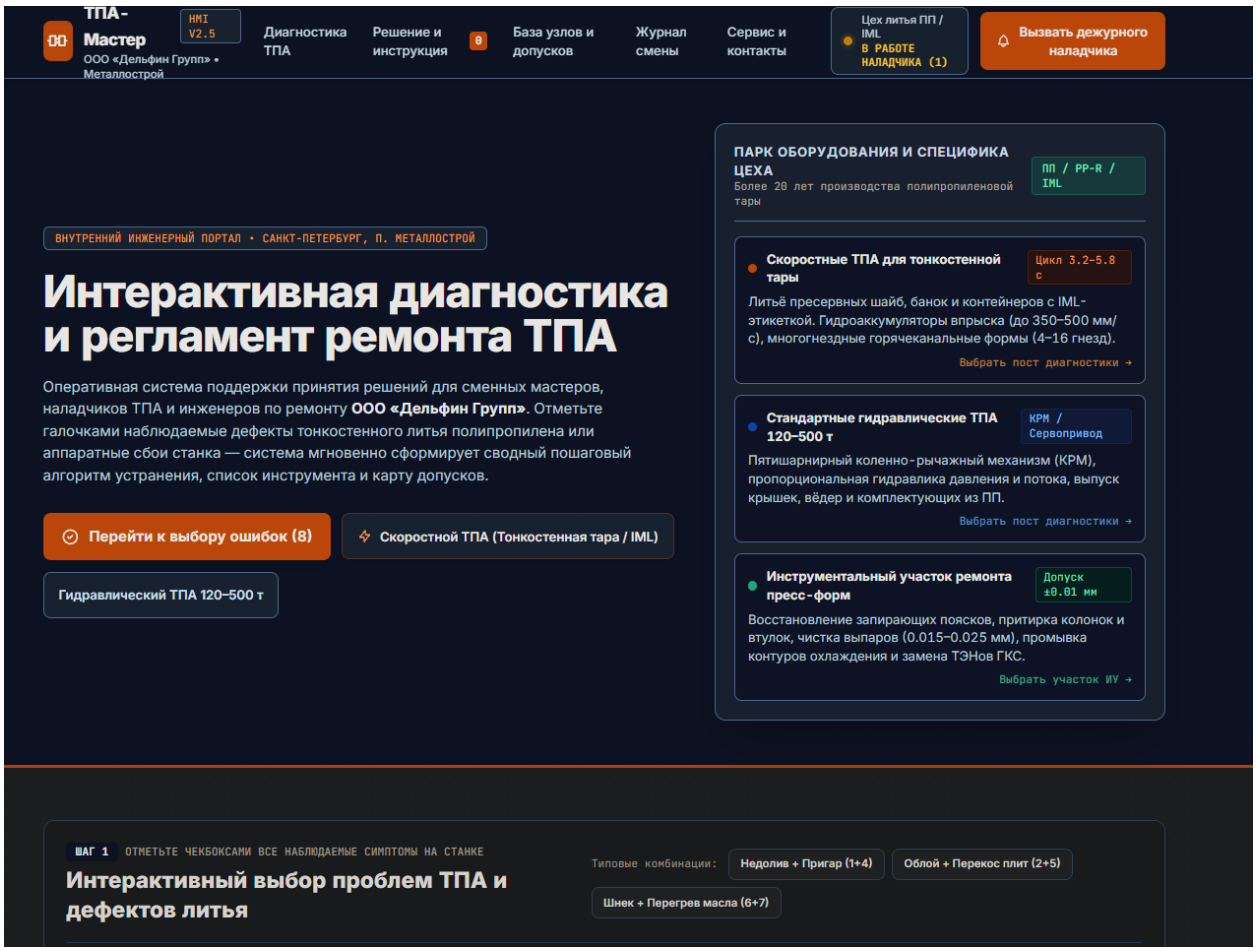


Рисунок 1 — Главный экран сайта диагностики ТПА «ТПА-Мастер: Дельфин Групп» после запуска через Live Server

4.1 Результаты проверки сайта

Я проверил сгенерированную страницу по трём основным пунктам из задания:

1) Визуальная часть: интерфейс выполнен в строгом индустриальном стиле без лишнего визуального шума. Текст и параметры отлично читаются благодаря высокому контрасту тёмно-графитовых панелей (#0F172A) и светлых карточек инструкций (#F8FAFC). Заголовки и шаги алгоритма ремонта набраны чётким гротеском Inter, а коды ошибок, давление (бар) и температуры зон нагрева (°C) выделены моноширинным шрифтом JetBrains Mono. Карточки неисправностей ТПА с чекбоксами аккуратно выстроены в ровную сетку с цветовыми индикаторами критичности (типовой дефект / аварийный сбой).

2) Функциональная часть: верхняя панель навигации зафиксирована при прокрутке и плавно переводит к нужному блоку. Работает переключение вкладок категорий («Классические ошибки литья», «Аппаратные и нетипичные сбои», «Пресс-формы») и мгновенный поиск по симптомам. При установке галочек (чекбоксов) на одной или сразу нескольких проблемах ТПА в блоке «Полное решение и инструкция» без перезагрузки

страницы формируется сводный пошаговый план ремонта с перечнем инструмента, мерами безопасности LOTO и нормативными параметрами. Кнопка записи сохраняет выбранный кейс в сменный журнал, а кнопка вызова открывает модальное окно заявки в ремонтную службу ООО «Дельфин Групп».

3) Адаптивность: я проверил страницу путём сжатия окна браузера, а также через панель разработчика Chrome DevTools (клавиша F12, режим Toggle Device Toolbar). На широком мониторе карточки ошибок ТПА стоят в 2–3 колонки рядом с панелью сводной инструкции, на планшете мастера перестраиваются в 2 колонки, а на экране смартфона выстраиваются в одну колонку с крупными областями нажатия для чекбоксов и рабочим мобильным бургер-меню (рисунок 2).

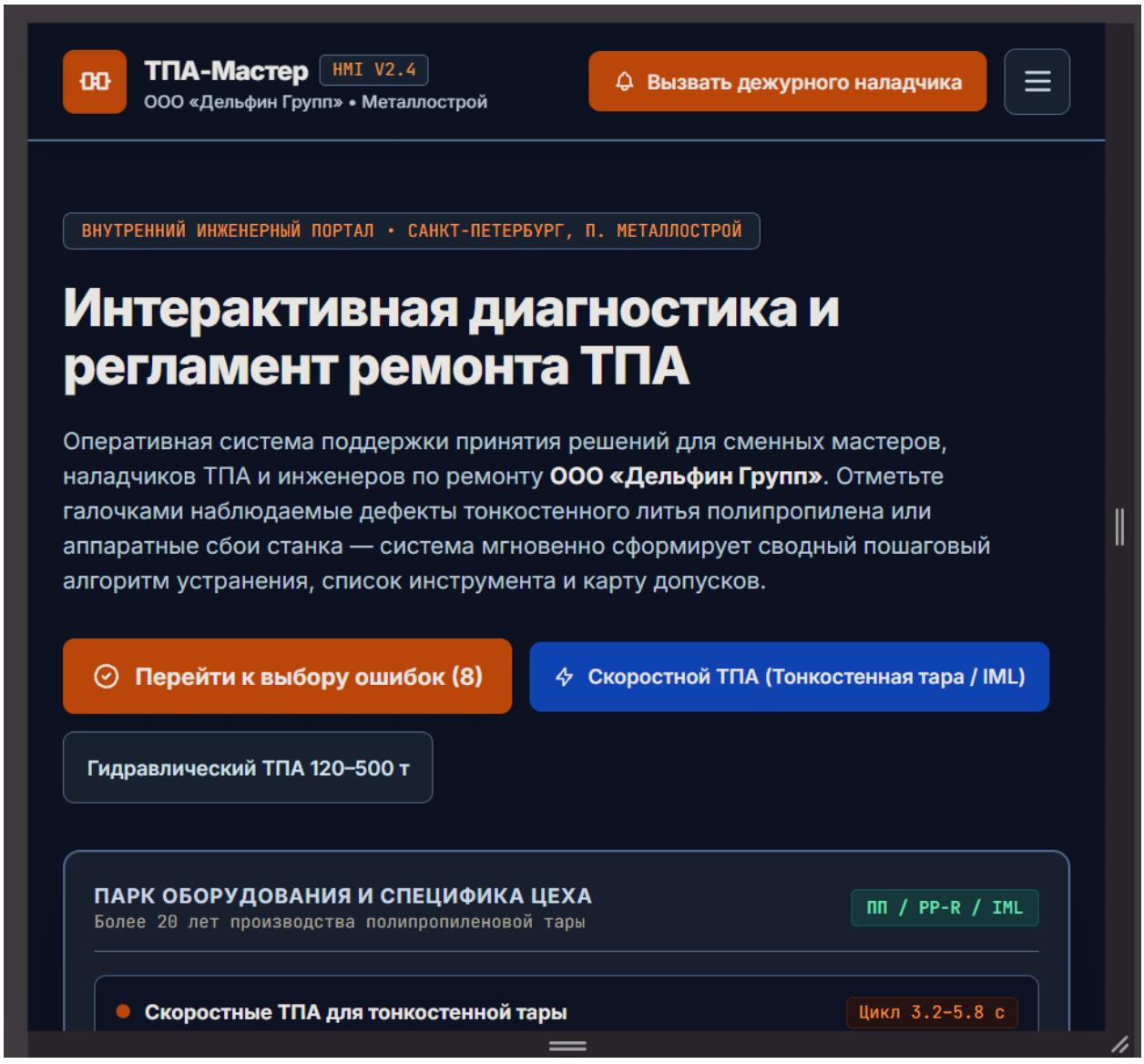


Рисунок 2 — Отображение интерактивного блока выбора ошибок ТПА и проверка мобильной вёрстки через DevTools

4.2 Сравнение с другой языковой моделью

Чтобы сравнить результаты, я отправил тот же самый промпт во вторую языковую модель — DeepSeek. Основные отличия между двумя результатами сведены в таблицу 1.

Таблица 1 — Сравнение результатов генерации сайта двумя языковыми моделями

Критерий	**Модель 1: Gemini**	**Модель 2: DeepSeek**
Внешний вид и удобство для мастера	Строгий индустриальный интерфейс, контрастные бейджи критичности, наглядные SVG-иконки узлов ТПА и удобные крупные чекбоксы.	Более простая табличная вёрстка со стандартными мелкими чекбоксами браузера, менее заметное разделение между симптомами и шагами ремонта.
Сетка карточек ошибок (8 неисправностей ТПА)	Ровная адаптивная сетка CSS Grid (по 2–4 карточки в ряд) с рабочей фильтрацией по вкладкам «Классические» / «Аппаратные» и поиском по коду ошибки.	Использован Flexbox с неравномерной высотой блоков, из-за чего при длинном описании симптомов сетка карточек разъезжается по высоте.
Генерация инструкции при выборе ошибок	При выборе нескольких ошибок динамически собирается сводная пошаговая инструкция с чек-листом шагов, списком инструмента, допусками температур ПП и записью в журнал.	Показывает подсказку только для одной последней нажатой ошибки, без объединения нескольких симптомов в общую инструкцию и без сохранения в журнал.

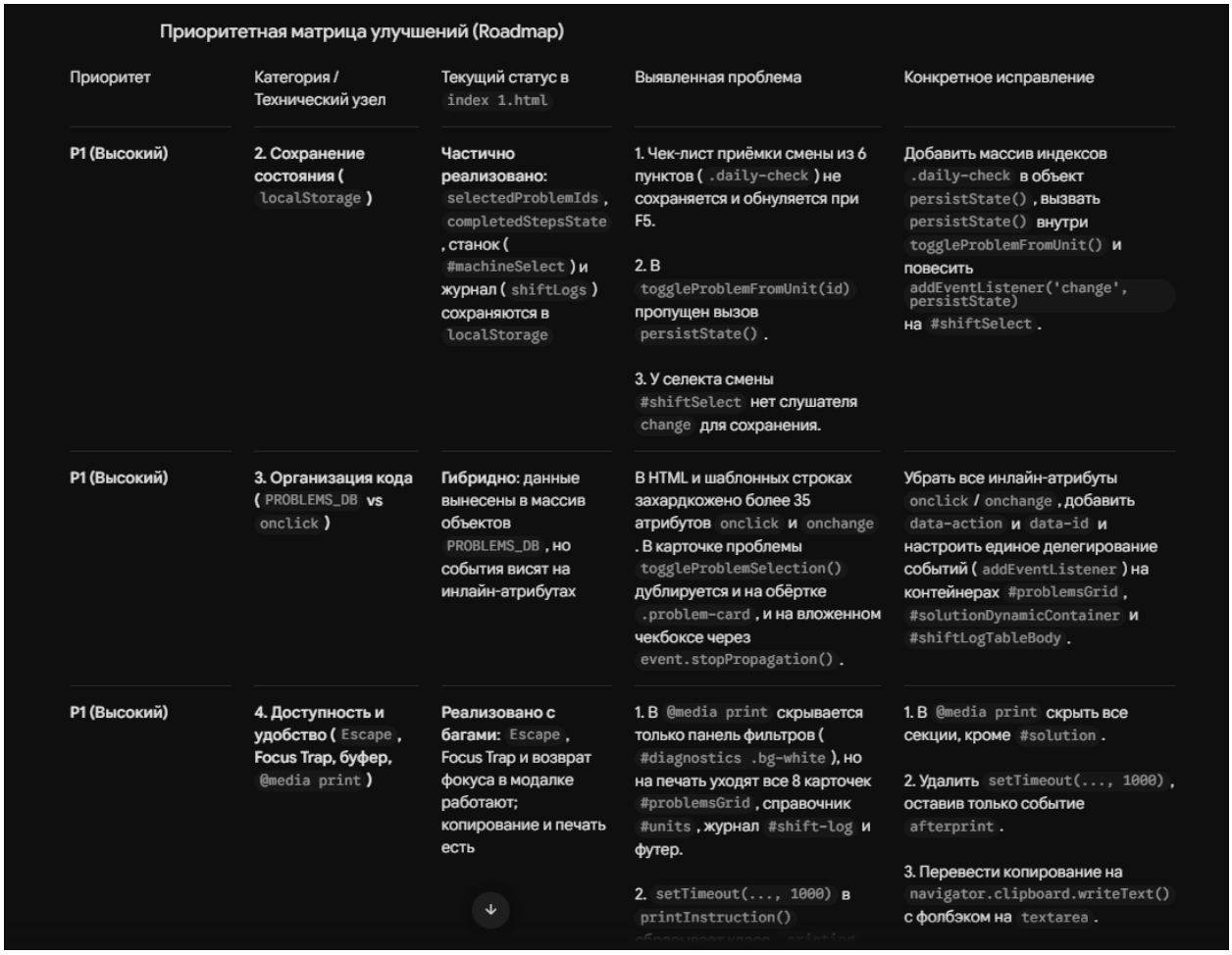
В итоге мне гораздо больше понравился вариант от модели Gemini, так как в нём сразу получился удобный для мастера интерфейс без лишних деталей, ровная сетка карточек неисправностей и полноценная логика формирования сводной инструкции при выборе сразу нескольких проблем на ТПА.

5 Анализ сайта с помощью LLM

Для проверки кода и поиска слабых мест я открыл новый чат с LLM и отправил туда свой файл кода (index 1.html) вместе с проверочным промптом из методички:

«Проанализируй мой лендинг как опытный UX/UI-дизайнер и frontend-разработчик. Оцени: визуальную иерархию; цветовую схему; типографику; навигацию; структуру страницы; CTA-элементы; адаптивность; качество текста; потенциальные проблемы UX; SEO; визуальную современность. Для каждого недостатка предложи конкретное

исправление. Не переписывай весь сайт. Сначала составь список приоритетных улучшений». Результат анализа кода в чат-боте показан на рисунке 3.



Приоритетная матрица улучшений (Roadmap)

Приоритет	Категория / Технический узел	Текущий статус в `index 1.html`	Выявленная проблема	Конкретное исправление
P1 (Высокий)	2. Сохранение состояния (`localStorage`)	Частично реализовано: `selectedProblemIds` , `completedStepsState` , станок (`#machineSelect`) и журнал (`shiftLogs`) сохраняются в `localStorage`	1. Чек-лист приёмки смены из 6 пунктов (`.daily-check`) не сохраняется и обнуляется при F5. 2. В `toggleProblemFromUnit(id)` пропущен вызов `persistState()` . 3. У селекта смены `#shiftSelect` нет слушателя `change` для сохранения.	Добавить массив индексов `.daily-check` в объект `persistState()` , вызвать `persistState()` внутри `toggleProblemFromUnit()` и повесить `addEventListener('change', persistState)` на `#shiftSelect` .
P1 (Высокий)	3. Организация кода (`PROBLEMS_DB` vs `onclick`)	Гибридно: данные вынесены в массив объектов `PROBLEMS_DB` , но события висят на инлайн-атрибутах	В HTML и шаблонных строках захардкожено более 35 атрибутов `onclick` и `onchange` . В карточке проблемы `toggleProblemSelection()` дублируется и на обёртке `.problem-card` , и на вложенном чекбоксе через `event.stopPropagation()` .	Убрать все инлайн-атрибуты `onclick` / `onchange` , добавить `data-action` и `data-id` и настроить единое делегирование событий (`addEventListener`) на контейнерах `#problemsGrid` , `#solutionDynamicContainer` и `#shiftLogTableBody` .
P1 (Высокий)	4. Доступность и удобство (`Escape` , Focus Trap, буфер, `@media print`)	Реализовано с багами: `Escape` , Focus Trap и возврат фокуса в модалке работают; копирование и печать есть	1. В `@media print` скрывается только панель фильтров (`#diagnostics .bg-white`), но на печать уходят все 8 карточек `#problemsGrid` , справочник `#units` , журнал `#shift-log` и футер. 2. `setTimeout(..., 1000)` в `printInstruction()`	1. В `@media print` скрыть все секции, кроме `#solution` . 2. Удалить `setTimeout(..., 1000)` , оставив только событие `afterprint` . 3. Перевести копирование на `navigator.clipboard.writeText()` с фолбэком на `textarea` .

Рисунок 3 — Результат анализа кода сервиса диагностики ТПА в чат-боте

Модель выделила следующие замечания и рекомендации по улучшению сайта:

1) SEO и технические мета-теги: в шапке документа не хватало разметки Open Graph для превью ссылки в рабочих мессенджерах, встроенной SVG-иконки сайта (favicon с изображением шестерёнки/станка) и микроразметки JSON-LD для производственно-сервисного предприятия ООО «Дельфин Групп».

2) Сохранение состояния диагностики (UX): при случайном обновлении страницы на планшете в цеху сбрасывались отмеченные мастером чекбоксы ошибок ТПА и прогресс выполненных шагов внутри инструкции по ремонту. Модель посоветовала сохранять текущие выбранные симптомы, отмеченные шаги чек-листа и записи сменного журнала в localStorage браузера.

3) Организация кода и базы знаний: вместо жёсткого дублирования HTML-разметки карточек неисправностей лучше хранить всю базу ошибок ТПА (симптомы, причины, инструмент, шаги ремонта и допуски) в едином структурированном массиве объектов

JavaScript, а обработчики выбора чекбоксов навешивать через делегирование событий (addEventListener), а не через атрибут onclick в разметке.

4) Доступность и работа с формой заявки и печатью: для модального окна вызова ремонтной бригады рекомендовано добавить закрытие по клавише Escape и удержание фокуса внутри окна (Focus Trap), а для блока готовой инструкции по ремонту — кнопку быстрой печати регламента (@media print) и копирования текста инструкции в буфер обмена.

Вывод по анализу: языковая модель качественно проверила код инженерного сервиса. Она обратила внимание на важные практические мелочи (сброс выбранных мастером ошибок при перезагрузке страницы, отсутствие стилей для печати листа ремонта и закрытия модального окна по Escape), которые критичны при реальном использовании сайта в цеху.

6 Доработка проекта с помощью ИИ-агента

Для выполнения последней части работы я использовал ИИ-агента SourceCraft Code Assistant. Сначала я попросил агента проанализировать текущий рабочий файл сайта (index 1.html / index.html) в папке проекта и составить план точечных исправлений, а затем поставил перед ним две небольшие задачи (как показано на рисунке 4, весь код при этом был оставлен в одном файле):

– *Задача 1:* сделать так, чтобы все отмеченные мастером чекбоксы ошибок ТПА, галочки выполненных шагов внутри инструкции по ремонту и записи «Сменного журнала мастера» автоматически сохранялись в localStorage браузера и не пропадали при перезагрузке страницы, а также добавить кнопку копирования готового регламента ремонта в буфер обмена для отправки в рабочий чат смены.

– *Задача 2:* добавить в блок контактов производственной площадки ООО «Дельфин Групп» (п. Металлострой, дорога на Металлострой, д. 5, лит. П) переключение на интерактивную карту проезда OpenStreetMap с быстрыми ссылками на Яндекс.Карты и 2ГИС, а также прописать в <head> теги Open Graph, встроенный индустриальный SVG-фавикон и микроразметку JSON-LD.

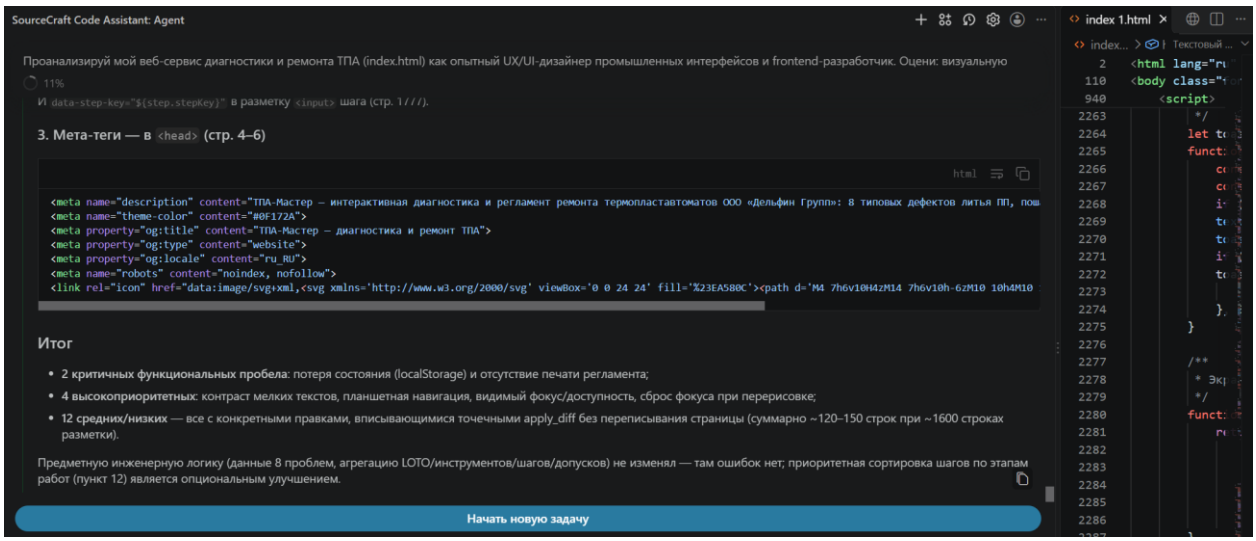


Рисунок 4 — Работа ИИ-агента SourceCraft Code Assistant в VS Code: анализ файла проекта, генерация блока мета-тегов и плана точечных правок

Наблюдения за работой агента: агент сам прочитал открытый в редакторе файл проекта, проанализировал все 1600+ строк разметки и скриптов, сформировал готовый блок мета-тегов с SVG-фавиконом для секции <head> и предложил точечные правки (apply_diff) без переписывания всей страницы целиком. Работать через агента оказалось намного удобнее, чем через обычный чат в браузере, потому что он видит весь код прямо в рабочей папке VS Code и избавляет от ручного поиска и копирования фрагментов кода. Разделение файла на отдельные index.html, styles.css и main.js я специально не делал на этом этапе, чтобы оставить эту задачу для следующей лабораторной работы.

Вывод

Выполняя эту лабораторную работу, я на практике попробовал два разных способа использования искусственного интеллекта при создании сайтов: работу через обычный диалоговый чат-бот с большой языковой моделью (LLM) и работу через ИИ-агента непосредственно внутри редактора кода VS Code.

Главный плюс обычных языковых моделей заключается в том, что они позволяют очень быстро получить готовую основу сайта. Если вручную писать разметку всех семи секций сервиса, подбирать контрастные цвета, настраивать адаптивную сетку для планшета и вручную верстать карточки неисправностей и пошаговые инструкции для каждого узла ТПА, на это уйдёт несколько вечеров. С помощью LLM рабочий прототип появился буквально за пару минут. При этом я убедился, насколько сильно итоговый код зависит от того, как написан промпт. Когда запрос составлен по структуре R-T-F-C (с указанием роли, контекста предприятия, подробного списка блоков и ограничений), модель сразу выдаёт логичную структуру страницы и удобный интерфейс для мастера. Кроме того, LLM отлично подходит для проверки уже написанного кода: в отдельном чате модель быстро подсказала, каких мета-тегов не хватает для SEO и как улучшить удобство сохранения выбранных чекбоксов.

ИИ-агенты в среде разработки (такие как SourceCraft Code Assistant) решают главную проблему обычных чат-ботов - оторванность от файлов проекта. Когда код сайта становится большим, постоянно копировать его целиком из браузера в редактор и обратно неудобно, к тому же можно случайно потерять часть вёрстки. Агент сам видит файлы в рабочей папке, читает нужные участки кода и точечно вносит исправления после подтверждения пользователем. Это сильно экономит время при доработке сайта и исправлении багов.

В то же время у этих технологий есть заметные ограничения. Во-первых, у любой модели ограничено контекстное окно: если проект большой или диалог слишком затянулся, нейросеть начинает путаться, забывать старые условия и быстрее расходовать лимиты запросов. Во-вторых, ИИ может допускать ошибки в коде - например, использовать нерабочие обработчики событий, забыть закрыть тег или сделать неровную сетку карточек (как получилось во второй сравниваемой модели, где блоки неисправностей ТПА разъехались по высоте и не объединялись в общую инструкцию). В-третьих, за действиями ИИ-агента обязательно нужно следить и проверять предлагаемые им изменения перед подтверждением, иначе он может удалить нужный кусок кода или изменить лишнее.

Во время выполнения лабораторной работы я столкнулся с тем, что на весь проект быстро переставало хватать доступных токенов. Из-за этого приходилось аккуратнее формулировать запросы и делить задачу на части, чтобы нейросеть не «забывала» контекст.

В итоге можно сказать, что LLM и ИИ-агенты - это отличные помощники, которые берут на себя рутинную вёрстку и написание шаблонного кода, но проверять вёрстку в браузере, тестировать адаптивность и контролировать итоговый результат всё равно должен сам разработчик.

Ответы на контрольные вопросы

1. Что такое искусственный интеллект? Искусственный интеллект (ИИ) - это направление компьютерных наук и совокупность технологий, которые позволяют программам выполнять задачи, обычно требующие человеческого мышления: обучаться на данных, распознавать образы, анализировать текст и принимать решения.

2. Что такое генеративный искусственный интеллект? Генеративный ИИ (Generative AI) - это вид искусственного интеллекта, который умеет создавать новый контент (текст, программный код, изображения, аудио или видео) на основе закономерностей, изученных во время обучения.

3. Что такое LLM? Большая языковая модель (Large Language Model, LLM) - это нейросетевая модель, обученная на больших объёмах текста и предназначенная для понимания, обработки и генерации естественного языка и программного кода.

4. Для чего используются большие языковые модели? Они используются для ответов на вопросы, написания и редактирования текстов, перевода, краткого пересказа документов, генерации и объяснения программного кода, поиска ошибок в коде и составления документации.

5. Что такое токен? Токен - это базовая единица текста (слово, часть слова или отдельный символ), на которые языковая модель разбивает входной и выходной текст при обработке.

6. Что такое контекст LLM? Контекст - это вся информация, которую видит модель при формировании ответа: сообщения пользователя, предыдущие ответы модели, прикреплённые файлы, куски кода и инструкции в текущем диалоге.

7. Что такое контекстное окно? Контекстное окно - это максимальный объём информации (в токенах), который модель может одновременно удерживать в памяти и учитывать при генерации ответа.

8. Почему большой программный код может быстро занимать контекстное окно? В программном коде очень много скобок, кавычек, отступов, служебных слов и длинных

путей, которые дробятся на большое число токенов. Если пересылать длинный файл кода в чате несколько раз подряд, контекстное окно быстро переполняется.

9. Что такое чат-бот? Чат-бот - это программный интерфейс (сайт или приложение), через который пользователь может общаться с языковой моделью в форме диалога.

10. Чем чат-бот отличается от LLM? LLM - это сама нейросеть (математическая модель), которая обрабатывает запрос и генерирует текст, а чат-бот - это оболочка (интерфейс с окном чата), передающая сообщения между пользователем и моделью.

11. Что такое промпт? Промпт - это текстовый запрос или инструкция, которую пользователь даёт нейросети для выполнения нужной задачи.

12. Почему качество промпта влияет на результат работы LLM? Если запрос написан слишком кратко и без деталей, модель не знает требований к дизайну, аудитории и структуре и начинает додумывать их сама. Чем точнее промпт, тем точнее результат.

13. Что означает принцип R-T-F-C? Это схема составления подробного промпта: R (Role) - роль, T (Task) - задача, F (Format) - формат ответа, C (Constraints) - ограничения (также часто отдельно выделяют Context - контекст).

14. Что такое Role? Role (Роль) - указание того, в качестве какого специалиста должна выступать модель (например, опытный frontend-разработчик).

15. Что такое Context? Context (Контекст) - описание предметной области, исходной ситуации, заказчика и целевой аудитории сайта.

16. Что такое Task? Task (Задача) - чёткое указание того, что именно нужно сделать и какие разделы или функции должны быть в результате.

17. Что такое Format? Format (Формат) - описание того, в каком виде нужно выдать ответ (например, один HTML-файл со встроенными стилями, таблица или список файлов).

18. Что такое Constraints? Constraints (Ограничения) - технические и визуальные правила, которые модель не должна нарушать (например, не использовать сторонние библиотеки, выдержать заданные цвета, обеспечить адаптивность).

19. Что такое AI-агент? AI-агент - это система на базе ИИ, которая может не только отвечать текстом, но и самостоятельно выполнять последовательность действий для решения задачи с помощью доступных ей инструментов.

20. Чем AI-агент отличается от обычного чат-бота? Чат-бот просто выдаёт текст в окне диалога, и человеку приходится самому копировать код в проект, а AI-агент имеет доступ к папке проекта и может сам читать, создавать и изменять файлы.

21. Какие инструменты может использовать AI-агент? Чтение и запись файлов, просмотр дерева папок проекта, поиск по коду, запуск команд в терминале, выполнение тестов и запуск браузера для проверки страницы.

22. Может ли AI-агент самостоятельно изменять файлы проекта? Да, агент может сам создавать новые файлы и редактировать существующий код в проекте (обычно запрашивая подтверждение у разработчика).

23. Почему AI-агент может быть эффективнее чат-бота при разработке программного обеспечения? Потому что он видит структуру всего проекта целиком, сам вносит правки в нужные места кода, запускает проверки и избавляет программиста от ручного копирования кода туда-сюда.

24. Какие риски возникают при использовании AI-агентов? Агент может неправильно понять задачу, изменить не тот файл, удалить нужный код, подключить несуществующую библиотеку или выполнить много лишних действий.

25. Почему действия AI-агента необходимо контролировать? Так как агент имеет прямой доступ к файлам проекта, без контроля со стороны человека он может сломать рабочую программу. Поэтому действует правило: ИИ выполняет работу, а человек контролирует результат.

26. В каких случаях достаточно обычной LLM? Когда нужно быстро сгенерировать простой одностраничный сайт в одном файле, составить промпт, придумать текст, объяснить кусок кода или провести разовую проверку файла.

27. В каких случаях целесообразно использовать AI-агента? Когда проект состоит из нескольких файлов (отдельно HTML, CSS, JS), нужно провести рефакторинг, найти ошибки сразу в нескольких модулях или пошагово дорабатывать проект прямо в VS Code.

28. Что такое итеративная работа с LLM? Это пошаговая разработка по циклу: обсуждение идеи - уточнение требований - генерация кода - проверка сайта в браузере - исправление ошибок и улучшение результата.

29. Почему нельзя полностью доверять результату, полученному от ИИ? Нейросеть может допускать синтаксические и логические ошибки, придумывать несуществующие библиотеки, вставлять битые ссылки на картинки или делать некорректную вёрстку на разных экранах.

30. Как можно проверить качество Web-сайта, созданного ИИ? Открыть сайт через Live Server в браузере, проверить читаемость текста, сетку и загрузку картинок, прокликать все ссылки и кнопки, сжать окно браузера и проверить мобильный вид через F12 (Toggle Device Toolbar), а также отправить код на проверку в новый чат с LLM.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГУАП. Методические указания по выполнению лабораторной работы № 4 по дисциплине «Информатика» [Электронный ресурс]. – Режим доступа: https://pro.guap.ru, свободный. – Загл. с экрана (дата обращения: 20.10.2026).

2. Tailwind CSS : официальная документация утилитарного фреймворка [Электронный ресурс]. – Режим доступа: https://tailwindcss.com, свободный. – Загл. с экрана (дата обращения: 20.10.2026).

3. Google Gemini : большая языковая модель : официальный сайт [Электронный ресурс]. – Режим доступа: https://gemini.google.com, свободный. – Загл. с экрана (дата обращения: 20.10.2026).

4. Visual Studio Code : редактор исходного кода : официальный сайт [Электронный ресурс]. – Режим доступа: https://code.visualstudio.com, свободный. – Загл. с экрана (дата обращения: 20.10.2026).

5. DeepSeek : большая языковая модель : официальный сайт [Электронный ресурс]. – Режим доступа: https://chat.deepseek.com, свободный. – Загл. с экрана (дата обращения: 20.10.2026).

6. SourceCraft Code Assistant : AI-помощник разработчика : официальный сайт [Электронный ресурс]. – Режим доступа: https://sourcecraft.dev, свободный. – Загл. с экрана (дата обращения: 20.10.2026).